

ECOM EXPERTS

Visor de Módulos

Manual de usuario y documento técnico

visor_modulos 1.0.0 · 03/09/2026

Contenido

1. Cómo se abre

2. Qué se ve

Los tipos que distingue

3. De dónde saca «para qué sirve»

4. Botones y atajos

5. Buscar

6. Preguntas rápidas

1. Ficheros

2. Cómo detecta cada cosa (escaner.py)

Clasificación

Metadatos del módulo PrestaShop

Resumen de los .md

3. La ventana (visor_modulos.py)

Dónde guarda

4. Construir el .exe

4 bis. El botón «Abrir con AntigraVity»

5. Tres trampas que costaron tiempo (y las pruebas que las cazan)

Programa de Windows que enseña, de una sola pantalla, **todos los módulos, extensiones y proyectos** de la carpeta `modulos IA` y **para qué sirve cada uno**.

No necesita PHP, ni navegador, ni servidor: es un `.exe` que se abre y ya está.

1. Cómo se abre

Doble clic en:

```
z:\modulos ps mios finalizados y ok\modulos IA\visor_modulos\Visor de Modulos.exe
```

Si se quiere tener a mano, botón derecho sobre el `.exe` → *Enviar a* → *Escritorio (crear acceso directo)*, o *Anclar a la barra de tareas*.

La primera vez tarda un par de segundos en leer la carpeta. A partir de ahí abre al instante mostrando la última lectura y **vuelve a revisar la carpeta por detrás**, así que las carpetas nuevas y los documentos `.md` nuevos aparecen solos. En la barra de abajo avisa de cuántos elementos ha leído y **cuáles son nuevos** desde la última vez.

2. Qué se ve

Zona	Qué hay
Arriba	El título, la carpeta que está mirando y los botones
Buscador	Escribe y la lista se filtra al momento
Tipo / Carpeta	Dos desplegables para quedarse solo con lo que interesa
Listado	Nombre · Tipo · Versión · Carpeta · Para qué sirve
Ficha (derecha)	Todo el detalle de lo que esté seleccionado
Abajo	Cuántos elementos hay y qué ha encontrado nuevo

Los tipos que distingue

- **Módulo PrestaShop** — la carpeta tiene un `<nombre>.php` con una clase que extiende `Module`.
- **Extensión de navegador** — tiene un `manifest.json` de Chrome o Firefox.
- **Módulo Odoo** — tiene `__manifest__.py`.
- **Tema WordPress** — tiene un `style.css` con `Theme Name:`.
- **Proyecto Python / Proyecto Node/Web / Proyecto** — el resto de cosas con código propio.

Cada tipo sale de un color distinto en el listado.

3. De dónde saca «para qué sirve»

Por este orden:

1. La **descripción del propio módulo**: lo que hay escrito en `$this->description` del fichero principal (lo mismo que se lee en el Gestor de módulos de PrestaShop). Para las extensiones, el `description` del `manifest.json`.
2. Si no hay descripción, el **primer párrafo de su documentación**, buscando primero en `manuales\<módulo>\` y después dentro de la propia carpeta. Da preferencia a `DOC-usuario.md`, `manual`, `LEEME` y `README`.
3. Si no hay ninguna de las dos cosas, lo dice claramente.

En la ficha de la derecha salen **las dos**: la descripción del código y, debajo, el resumen del documento del que lo ha sacado (con el nombre del fichero).

4. Botones y atajos

	Qué hace
Actualizar (o <code>F5</code>)	Vuelve a leer la carpeta entera
Cambiar carpeta...	Apunta el programa a otra carpeta cualquiera; se recuerda
Exportar listado	Guarda lo que se está viendo (respeta los filtros) en <code>.md</code> o en <code>.csv</code>
Abrir carpeta (o doble clic en la fila)	Abre la carpeta en el Explorador
Abrir documento	Abre el <code>.md</code> seleccionado de la lista de documentos
Copiar ficha	Copia al portapapeles el resumen de ese módulo
Abrir con AntigraVity	Abre esa carpeta en AntigraVity, en una ventana nueva
<code>Ctrl+F</code>	Salta al buscador
<code>Esc</code>	Vacía el buscador
Clic en una cabecera	Ordena por esa columna; otro clic, al revés

En la lista de documentos, **doble clic** abre el `.md` con el programa que se tenga asociado.

5. Buscar

El buscador mira a la vez en el nombre, el tipo, el título, la descripción, el resumen del manual, la carpeta, el autor y los hooks. Se pueden poner **varias palabras** y las busca todas: `whatsapp pedido` , `stock combinaciones` , `displayHeader` .

6. Preguntas rápidas

El botón de Antigravity no encuentra el programa. Lo busca solo en

`%LOCALAPPDATA%\Programs\Antigravity IDE\bin\` y en las rutas habituales de Archivos de programa. Si no lo encuentra, pregunta dónde está y se queda con la ruta para siempre. Abre siempre una **ventana nueva** con esa carpeta, aunque ya tengas abierta la carpeta que la contiene.

¿Modifica algo de mis módulos? No. Solo lee. No escribe ni un byte dentro de las carpetas de los módulos.

¿Dónde guarda sus cosas? En `%LOCALAPPDATA%\VisorModulos\` : `config.json` (la carpeta elegida) y `cache.json` (la última lectura, para que abra al instante). Se pueden borrar sin problema.

Sale un módulo dos veces. Es que está de verdad dos veces: normalmente una copia en `old modules` o en `modulos convertidos a 8.2` . La columna **Carpeta** dice dónde está cada una.

Un módulo aparece sin descripción. No tiene `$this->description` con texto ni ningún `.md` . Se arregla escribiendo la descripción en el módulo o poniéndole un `manuales\<módulo>\DOC-usuario.md` .

No aparece una carpeta que sí tengo. Se listan las cosas con código propio. Las carpetas que solo tienen documentos o solo subcarpetas se consideran contenedores (como `manuales` o `capturas`) y no salen en la lista, pero sí se entra en ellas a buscar módulos.

Visor de Módulos — documento técnico

Programa de escritorio para Windows, en Python 3 + Tkinter, empaquetado en un `.exe` de un solo fichero con PyInstaller. **Cero dependencias en ejecución:** el `.exe` no necesita Python instalado ni nada más.

Código en `modulos IA\visor_modulos\`.

1. Ficheros

Fichero	Qué es
<code>escaner.py</code>	Recorre la carpeta y construye las fichas. Solo biblioteca estándar
<code>visor_modulos.py</code>	La ventana (Tkinter/ttk) y toda la interacción
<code>generar_icono.py</code>	Dibuja <code>visor_modulos.ico</code> con Pillow (solo para construir)
<code>construir_exe.py</code>	Genera el <code>.exe</code> ; antes pasa la prueba de la ventana
<code>prueba_visor.py</code>	Prueba de humo: abre la ventana y pulsa todos los botones
<code>prueba_exe.py</code>	Lanza el <code>.exe</code> ya construido, comprueba que la ventana aparece y lo mata por su PID
<code>Visor de Modulos.exe</code>	El entregable (unos 11 MB)

2. Cómo detecta cada cosa (`escaner.py`)

`escanear(raiz)` hace un `os.walk` con poda hasta `PROFUNDIDAD_MAX = 6` y devuelve una lista de diccionarios ordenada por nombre.

Clasificación

`clasificar(carpeta, ficheros, subdirs)` mira los ficheros de la carpeta:

Detecta	Por qué
Módulo PrestaShop	Existe <code><nombre_carpeta>.php</code> . Se confirma con la regex <code>class X extends Module\ PaymentModule\ CarrierModule\ ModuleGrid\ ModuleGraph</code> ; si no la cumple, se degrada a <i>Proyecto</i>
Extensión de navegador	<code>manifest.json</code> con <code>"manifest_version"</code>
Módulo Odoo	<code>__manifest__.py</code>
Tema WordPress	<code>style.css</code> con <code>Theme Name:</code>
Proyecto Node/Web	<code>package.json</code>
Proyecto Python	<code>requirements.txt</code> , <code>pyproject.toml</code> o cualquier <code>.py</code>
Proyecto	Hay ficheros de código (<code>EXT_CODIGO</code>)

Dos reglas que evitan la mayor parte del ruido:

1. Los cuatro tipos **identificables por manifiesto** valen a cualquier profundidad. Los genéricos (*Proyecto*, *Proyecto Python*, *Proyecto Node/Web*) **solo se listan en los dos primeros niveles**, no se listan si el nombre está en `CARPETAS_INTERNAS` (`classes` , `controllers` , `views` , `backend` , `api` ...) y **no se listan si su carpeta padre ya se listó como proyecto**: las subcarpetas de un proyecto no son proyectos aparte.
2. Una carpeta que solo tiene documentos o solo subcarpetas **no se lista**: es un contenedor (`manuales` , `capturas` , `old modules`). Se entra en ella a buscar, pero no aparece como una cosa más.

Dentro de un módulo ya detectado se poda el recorrido (`dirs[:] = []`): no hay módulos dentro de módulos.

Metadatos del módulo PrestaShop

`leer_modulo_php()` no ejecuta PHP: aplica una regex sobre las asignaciones `$this->name|version|author|tab|displayName|description|ps_versions_compliancy` y saca los literales de la expresión, con estas particularidades:

- Funciona igual con `$this->l('...')` , con `$this->trans('...', array(), 'Modules.X.Admin')` y con la cadena a pelo.
- **Los dominios de traducción se descartan** (`Modules.Xxx.Admin` , `Admin.*` , `Shop.*`), que si no aparecerían como si fueran la descripción.
- Las **concatenaciones** (`'a' . 'b'`) se unen; las descripciones largas partidas en varias líneas salen enteras.
- De `ps_versions_compliancy` saca «desde X hasta Y».
- Cuenta los hooks distintos de los `registerHook(...)` .

Resumen de los `.md`

`resumir_md()` coge el **primer párrafo de verdad**: salta el título, los bloques de código (```), las tablas, las listas, las citas y las imágenes; limpia negritas, código y enlaces Markdown; y corta por palabra a 700 caracteres.

Los documentos se ordenan por `PRIORIDAD_DOCS` (`DOC-usuario`, `guia-usuario`, `manual`, `LEEME`, `README`, `DOC-tecnico` ...) y se busca primero en `manuales\<módulo>` y después dentro de la propia carpeta, hasta dos niveles.

`para_que_sirve(ficha)` es la que decide qué texto se enseña: `descripcion` → `resumen` → `titulo` → aviso de que no hay nada.

3. La ventana (`visor_modulos.py`)

- `Tk` con tema `clam` y estilos propios; arranca maximizada (`state('zoomed')`) y con **DPI awareness** (`SetProcessDpiAwareness(1)`), sin la cual Windows la escala y sale todo borroso.
- **El escaneo va en un hilo** y el resultado vuelve al hilo de la interfaz con `self.after(0, ...)`; tocar widgets desde el hilo secundario cuelga Tk.
- Arranque en dos tiempos: pinta la caché (`%LOCALAPPDATA%\VisorModulos\cache.json`) y a los 150 ms lanza el escaneo real. Al terminar compara las rutas de antes con las de ahora y **anuncia lo nuevo** en la barra de estado.
- El filtrado es en memoria sobre la lista completa: buscador (todas las palabras tienen que aparecer), tipo, carpeta y ordenación por cualquier columna.
- Los `wraplength` de la ficha **se recalculan con el ancho real del panel** (`_ajustar anchos`, atado al `<Configure>` del panel derecho): con un valor fijo el texto se corta o deja medio panel vacío según la pantalla.

Dónde guarda

```
%LOCALAPPDATA%\VisorModulos\config.json    la carpeta elegida
%LOCALAPPDATA%\VisorModulos\cache.json     la última lectura
```

Nada se escribe dentro de las carpetas revisadas: el programa **solo lee**.

4. Construir el `.exe`

```
python -m pip install pyinstaller pillow
cd "z:\modulos ps mios finalizados y ok\modulos IA\visor_modulos"
python construir_exe.py
```

`construir_exe.py` genera el icono si falta, **ejecuta** `prueba_visor.py` **y aborta si falla**, y luego llama a PyInstaller con `--onefile --windowed`, el icono incrustado y `--distpath` en la propia carpeta. Los ficheros intermedios van al temporal y se borran.

Para comprobar el `.exe` ya construido:

```
python prueba_exe.py
```

4 bis. El botón «Abrir con Antigravity»

`buscar_antigravity()` mira, **por este orden**, el guion de la CLI (`<Programas>\Antigravity IDE\bin\antigravity-ide.cmd`) y solo después el `.exe` de la aplicación; si no encuentra nada, prueba el `PATH` y, en último término, le pregunta al usuario y guarda la ruta en `config.json`.

El lanzamiento es:

```
antigravity-ide.cmd --new-window --folder-uri "file:///z:/.../ecom_quotes"
```

Dos cosas que hay que hacer así y no de otra manera:

- **Antigravity IDE.exe <carpeta> no hace nada** cuando el programa ya está abierto: el `.exe` es el lanzador de la aplicación, no la CLI. Comprobado: cero ventanas nuevas en 25 segundos, ni con la carpeta ni con `-n`.
- **Pasar la ruta a secas tampoco vale.** Si esa carpeta está dentro de un proyecto que ya está abierto (y todos los módulos están dentro de `modulos IA`), Antigravity se limita a enfocar esa ventana, y desde fuera parece que el botón no funciona. Con `--folder-uri` abre siempre la carpeta pedida, en su ventana.

La prueba real está en `python prueba_visor.py --antigravity`: lanza Antigravity de verdad, espera a que aparezca una ventana **nueva** y se la cierra con `WM_CLOSE`. La prueba normal solo comprueba el cableado con un espía.

5. Tres trampas que costaron tiempo (y las pruebas que las cazan)

El `.exe` de un solo fichero arranca un proceso hijo. La ventana **no** pertenece al PID que devuelve `Popen`: el bootloader de PyInstaller se descomprime y lanza otro proceso. Buscar la ventana por ese PID no encuentra nada y parece que el programa no arranca. `prueba_exe.py` busca una ventana **nueva por su título** y al terminar mata el árbol entero (`taskkill /F /T /PID`) más el nombre de imagen, porque si no quedan instancias colgadas. Nunca `wmic`: no existe en Windows 11.

Un botón mal conectado no lo caza ningún análisis estático. Por eso `prueba_visor.py` abre la ventana de verdad y pulsa todo: ordenar por cada columna en los dos sentidos, los tres filtros, una búsqueda sin resultados, recorrer 20 fichas, y los seis botones (con `filedialog` y `abrir_en_windows` sustituidos para que

no abra ventanas del sistema). Comprueba además que más del 90 % de los elementos tienen descripción, que es lo único que hace útil al programa.

Un espía que sustituye a la función también engaña a tu propia prueba. `pruebavisor.py` cambia `vm.abrir_con_antigravity` por un espía para pulsar el botón sin abrir nada. La comprobación real llamaba a esa misma referencia y, por tanto, al espía: daba «no abre» siempre. La función de verdad se guarda en `LANZAR_ANTIGRAVITY` **antes** de poner el espía.

La captura de `prueba_exe.py` usa `PrintWindow` con `PW_RENDERFULLCONTENT` : así fotografía **solo esa ventana** sin traerla al frente ni interrumpir al usuario. Con `ImageGrab.grab(caja)` se fotografía lo que haya encima.